



ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF BIOMEDICAL ENGINEERING

**PORTFOLIO: COMPILED LIST OF ENGINEERING PROJECTS PROTOTYPED
DURING TENURE AT BIOELECTRICAL ENGINEERING LABORATORY (BEELAB)**

Supervisor Name: Dr. Mehmet Yuksekkaya

Student Name: Augusta Nwosu



Table of Contents

PREFACE	5
Project Title: Patient Strain Detection and Web-Based Monitoring System	6
Abstract.....	6
Introduction	6
Methodology.....	7
System Architecture And Data Flow	7
Clinical Dashboard And Alert UI.....	8
Hardware And Electronics Implementation.....	9
Bill of materials (electronics)	10
PCB Design And Verification	11
Embedded firmware (Arduino IDE).....	13
Design.....	13
Front End Services.....	13
Backend services	13
WebSockets.....	13
MongoDB logging.....	14
Analysis	14
Conclusion.....	14
Appendix A: ESP32 firmware	14
Appendix B: HX711 calibration sketch	17
Appendix C: Prototyped Handle	18
Project Title: Fatigue Detection System for Unmanned Aerial Vehicle (UAV) Pilot	19
Abstract.....	19
Introduction	19
Methodology.....	19
1. System requirements and interface.....	19
2. Readout circuit and signal conditioning.....	19
3. PCB design and component finalization	20
Schematic And PCB Design	21
4. PCB bring-up, testing, and verification	22
Analysis	22
Conclusion.....	22
Project Title: Non-Contact Type Clinical Thermometer	23



Abstract.....	23
Introduction	23
Project role and scope of electrical engineering work	23
Methodology.....	23
System architecture overview	23
Electronics implementation details	24
Component selection and configuration item list	25
PCB Design And Verification	25
Firmware-hardware interaction (summary)	26
Verification and testing approach.....	26
Analysis	26
Conclusion.....	26
Project Title: Smart Wearable Rehabilitation Glove with Flexible Capacitive Sensors for Physiotherapy Progress Monitoring	27
Abstract.....	27
Introduction	27
Methodology.....	27
System Overview.....	27
Engineering Phases	27
Electronics and Hardware Components (Planned).....	28
PCB and Hardware Implementation	28
Roles, Duties, and Contributions	28
Analysis	28
Success Criteria	28
Risks and Mitigation.....	29
Conclusion.....	29
Project Title: Design And Test of a Phonocardiogram System.....	30
Abstract.....	30
Introduction	30
Methodology.....	30
System overview	30
Electronics implementation	30
Schematic And PCB Design	30
Microphone options and coupling.....	31



Components and bill of materials	32
Role Summary	33
Assembly and verification workflow	33
Analysis	33
Signal capture and qualitative validation.....	33
Electronics-focused observations	34
Conclusion.....	34



PREFACE

This is a compiled list of project worked on by Augusta Nwosu under BEELAB. The purpose of this document is to showcase and detail the responsibilities undertaken by the aforementioned individual across each project. It is not to be considered as the official report or final documentaion for any of these projects. Instead, it serves as a concise record of contributions, roles, and key contibutions completed within the scope of each work. Official project reports, supporting design files, and full experimental documentation can be provided upon direct request.

Project 1 - Title: Patient Strain Detection and Web-Based Monitoring System

Contribution: Electronics, Embedded Firmware, Real-Time Web Dashboard, and Clinical Trial Leadership

Client/Partner: Ankara University - Department of Nursing

Design Links: [PATIENT STRAIN DETECTION SYSTEM](#) || [BACKEND SERVER](#)

Abstract

This project delivers a connected strain detection system designed for restrained psychiatric patients, with the goal of reducing harm by providing real time alerts to clinicians and nursing staff. A strap connected to the strain module measures pulling force using a load-cell front-end and an ESP32 controller. The alert triggers on-device audiovisual alarms and posts event data over HTTPS to a Node.js backend. The backend logs each alert to MongoDB and pushes real-time updates to a React/TypeScript (Next.js) clinical dashboard via WebSockets, enabling staff to view current risk status, recent attempts, and patient history. The author led a four-person team to design the end-to-end electronics stack (from the readout circuit, component selection, PCB design to testing), embedded firmware, full-stack web implementation, database integration, and ongoing clinical trial leadership.

Introduction

In acute psychiatric care, physical restraints may be used to prevent immediate harm. These restraints still cause safety risks when a distressed patient pulls, twists, or fights against cuffs and straps. Excessive pulling can cause soft-tissue injury, nerve compression, circulation compromise, skin breakdown, or escalation into self-harm attempts. This often occurs within a short window where the staff response time is the deciding factor. This project was developed for the Department of Nursing, Ankara University to provide an early-warning layer that detects clinically meaningful pulling forces on restraint cuffs and immediately alerts attending clinicians and nurses, reducing the time between a strain event and intervention.



Figure 1: Patient Restraint Cuff and Straps

The system combines a strain-sensing restraint module with a real-time clinical dashboard. On the patient side, a load-cell-based strain mechanism measures pulling force and triggers local alerts (buzzer/LEDs) while simultaneously transmitting an event to a server. On the clinical side, a web dashboard presents live status (“WebSocket active”), identifies the most at-risk patient, and lists recent events with timestamps, while also maintaining a structured patient history for review. This dual approach supports both immediate intervention (real-time warnings) and clinical accountability (audit logs and trend history).

A key design goal is to fit the realities of ward workflow. The nurses must be able to understand the alert at a glance. For that reason, the project emphasizes a clearly defined threshold-based trigger for high-risk pulling, rapid event delivery via WebSockets, and MongoDB logging to preserve every alert for clinical trials, evaluation, and post-incident review.

Methodology

System Architecture And Data Flow

Phase 1 — Sensing and On-Device Processing (ESP32 + Readout Circuit)

The strain strap is instrumented using a load cell conditioned by the HX711 amplifier. converts mechanical pull into a stable digital signal. The ESP32 samples the HX711 output, applies calibration (zero offset and scale factor), and computes force in Newtons in real time. When the measured force crosses the trigger threshold, the device immediately initiates local safety feedback while preparing the event details for transmission. An ADC pin is dedicated for battery percentage estimation to keep the device reliable.

Phase 2 — Network Access and Secure Telemetry Transport (Wi-Fi + HTTPS)

After boot, the ESP32 connects to the hospital network (enterprise Wi-Fi) and maintains a stable connection for continuous monitoring. Upon a trigger event, the device formats the alert as JSON and sends it to the server over HTTPS using a secure client configuration. This transmission directly from the ESP32 ensures that alerts reach the central system even when multiple devices are operating and connected to the frontend simultaneously.

Phase 3 — Backend Ingestion and Business Logic (Node.js API Layer)

The Node.js backend acts as the control hub. It receives alert messages, validates the payload, timestamps the event, and standardizes it into a server-defined schema. Once the data is validated, the backend applies basic business logic. It computes the number of attempts, updates the most at risk patient indicator, and prepares a broadcast-ready event object. This layer also has endpoints for the dashboard to fetch summaries and historical records, separating real-time alerting from on-demand analytics queries.

Phase 4 — Real-Time Alert Distribution (WebSockets)

Immediately after processing the data, the backend pushes the alert to connected clinician dashboards using WebSockets, enabling near-instant notification without page refresh. The WebSocket channel carries the details of the live event such as the patient name, strain magnitude,

time, alert state, updating all subscribed clients simultaneously. This is what drives the “WebSocket active” status indicator and ensures nurses and clinicians see new strain events as they happen, minimizing response time in high-risk scenarios.

Phase 5 — Persistent Storage and Audit Trail (MongoDB Logging)

In parallel with real-time broadcasting, every alert is written to MongoDB as a permanent log entry. This creates an auditable history for clinical evaluation and trial reporting. Each document stores the key event fields; the patient identifier, reading, timestamp, and server status, enabling structured queries such as “most recent attempts,” per-patient history, and weekly attempt counts to review the patient trend. This storage layer makes the system clinically useful beyond real-time response by supporting reviews, trend analysis, and evidence generation during trials.

Phase 6 — Clinician Interface and Decision Support (Figma Design to React/TypeScript + Next.js Front End)

The clinician dashboard is designed in Figma and implemented in React/TypeScript with Next.js. It involves both the WebSocket stream, live alerts and backend APIs, history and summaries. The interface prioritizes immediate safety: it highlights the most at-risk patient, lists recent attempts with timestamps, and provides expandable per-patient history panels for context. When a new alert arrives, the UI transitions into an attention-grabbing warning state, ensuring clinicians are informed even during multitasking.

Clinical Dashboard And Alert UI

The web interface was implemented in React/TypeScript using Next.js. It displays a WebSocket connection indicator, the most at-risk patient summary, a table of recent attempts, and expandable patient history entries. A dedicated warning screen is used to visually escalate active events and can be paired with browser or OS notifications in clinical workflows.

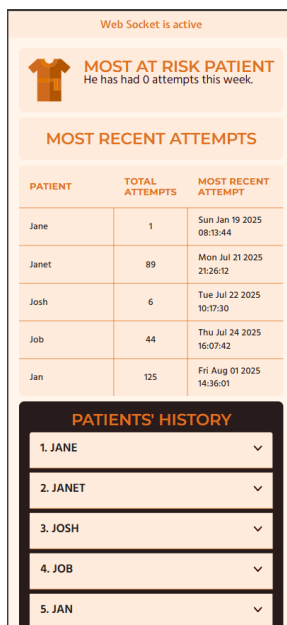


Figure 2: Dashboard view showing WebSocket status, risk summary, and recent attempt history || Figure 3: Warning screen used to alert clinicians/nurses during a pull/strain event (example UI state).

Hardware And Electronics Implementation

1) System Hardware Overview

The wearable strain detector is built around a load cell + HX711 instrumentation stage + ESP32 controller, with local audible and visual alarms, and a small OLED status display. The electronics are consolidated on a custom PCB, designed by the contributor that provides: stable sensor interfacing, power distribution, protection, capacitive decoupling, and reliable connectors for the strain module, battery, buzzer, LEDs.

2) Sensing Hardware: Load Cell + HX711 Front-End

- Sensor element: a load cell mechanically integrated into the strap so pulling force produces measurable strain.
- Signal conditioning - ADC: HX711 (24-bit ADC designed for load cells).
- Wiring is routed to a dedicated LOAD_CELL header on the PCB (A+, A-, E+, E-), providing a clean interface for excitation and differential sensing.
- The ESP32 reads HX711 via two digital lines: DT (data) and SCK (clock) connected to GPIO4 and GPIO2 respectively.
- Noise control: Local 100 nF decoupling capacitors are placed near supply pins/critical rails on the PCB to suppress switching and Wi-Fi transients that can otherwise corrupt low-level strain measurements.

3) Microcontroller and Control Electronics: ESP32 Core

Controller: ESP32 (mounted/connected via pin headers on the main PCB). It interfaces the HX711 (DT/SCK), drives alarm outputs (buzzer + LEDs), reads battery voltage via ADC. supports I²C expansion pins (SDA/SCL header on the PCB) for peripherals and debug extensions.

Pin mapping from your firmware:

HX711: SCK = GPIO2, DT = GPIO4
LEDs: GPIO33, GPIO25
Buzzer: GPIO14
Battery sense: GPIO12
Button: GPIO13

4) Local Alert Hardware: Buzzer + Dual LEDs + Button

This subsystem ensures the device is useful even if there is an attendant nearby. There is a dedicated BUZZER footprint exists on the PCB. The buzzer line includes a series resistor footprint (BUZ_RES ~10 kΩ as labeled on the board) to shape/limit drive depending on buzzer type and board revision. Firmware drives an audible alarm pattern when force exceeds threshold (rapid tone/noTone cycling). There are also dual LED indicators. Two separate LEDs are driven from GPIO33 and GPIO25. The PCB includes current-limiting resistors. These provide immediate visual confirmation of an active strain event and device activity.

There is also a manual trigger button. This button input is read on GPIO13 and used as a test/override. The PCB includes a button pull resistor footprint to enforce stable logic levels and prevent floating inputs.

5) Display Hardware: OLED Module (I²C)

Display: 128×32 OLED using the SSD1306 driver.



Bus: I²C through the ESP32's SDA/SCL pins (the header is provided on the PCB labeled SDA/SCL/VCC/GND).

Purpose in the device:

- Battery status display (percentage estimation).
- Basic system feedback (ready state, messages, or short alerts depending on final UI choice).

6) Power System: Battery + Charging + Regulation Path

Battery input: The PCB includes a BATTERY header (B+ / B-).

Charging module footprint: A dedicated area labeled for a TP4056-based Li-ion charger module (common for single-cell LiPo/Li-ion charging).

Power control: A board-mounted switch footprint provides a hardware on/off point, separating storage and charging from active operation.

Battery monitoring circuit

Battery voltage is routed through a voltage divider to an ADC pin (GPIO12) so the ESP32 can estimate battery percentage.

The firmware scales ADC reading with a multiplier ($\times 2$), consistent with a voltage divider intended to bring battery voltage into ADC-safe range. Firmware maps voltage to percentage.

7) PCB Implementation and Connectors

The PCB layout consolidates the full device wiring into labeled blocks to reduce assembly errors and improve field reliability. The components and pins are written clearly on the device to improve independent, stand-alone functionality.

8) Calibration Hardware Procedure (How the Electronics Enable It)

The design supports calibration through:

- Stable HX711 wiring and decoupling (reduces drift/noise),
- Firmware-assisted calibration workflow (serial tuning of calibration factor),
- Zero offset capture (your ZERO_OFFSET) and scaling (SCALE_FACTOR) to convert raw ADC counts $\rightarrow$ Newtons.

Bill of materials (electronics)

Component	Qty	Role / Notes
ESP32 development board	1	Primary microcontroller; WiFi + GPIO interface for sensor, OLED and alarms.
HX711 load cell amplifier/ADC	1	24-bit ADC and instrumentation front-end for strain/load measurement.
Load cell / strain sensor module	1	Mechanical-to-electrical transducer mounted to restraint strap/fixture.
OLED display (SSD1306, 128x32,	1	Local status display

I2C)		(battery/alerts).
Passive buzzer	1	Audible alarm for on-device alerting.
LED indicators	2	Visual alarm + status outputs.
Tactile push-button	1	Manual test/trigger and local interaction.
On/off slide switch	1	Power control.
LiPo battery (single-cell)	1	Portable power source.
TP4056 charging module footprint	1	Charging/protection for LiPo battery.
Resistors: 10 kOhm	3+	Pull-up/down and divider networks (as labelled on PCB).
Resistors: 220 Ohm	3+	LED current limiting (as labelled on PCB).
Capacitors: 100 nF	3	Supply decoupling / noise suppression (C1-C3 on PCB).
Headers/connectors	Assorted	Load cell, battery, programming and I/O connectors.
PCB (custom)	1	Carrier board integrating sensing, power, alarms and connectivity.

PCB Design And Verification

PCB design work included schematic capture, component footprint selection, routing, ground/power planning, and fabrication-ready checks. After assembly, basic bring-up tests confirmed power integrity, sensor acquisition, alarm outputs, and stable network communication under event-triggered load.

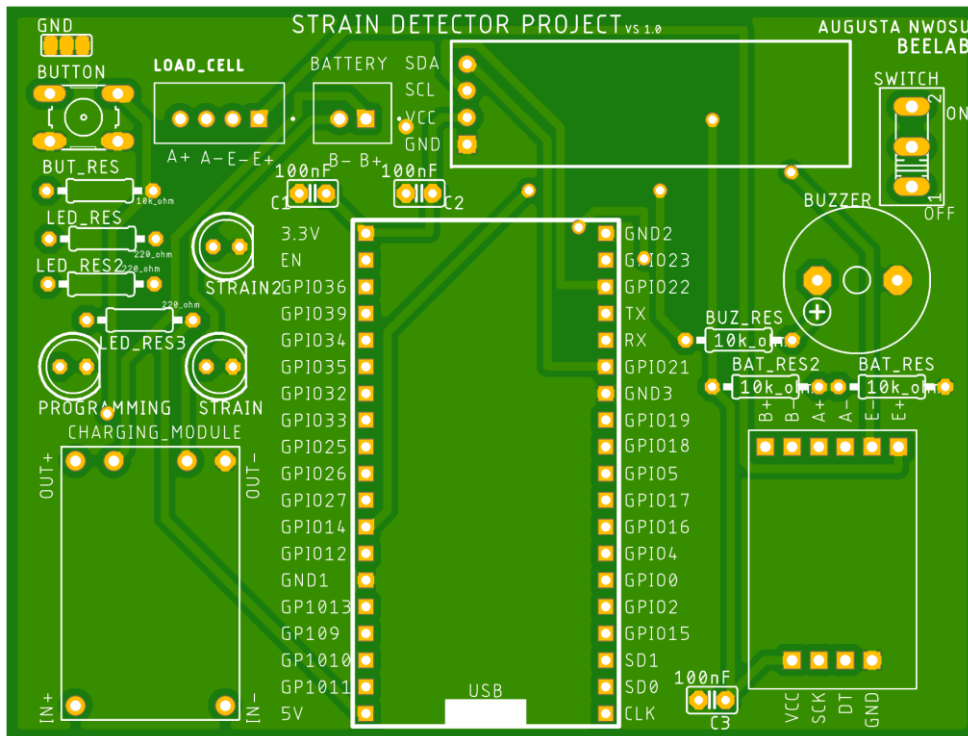


Figure 3: PCB render and component placement overview (Strain Detector Project v1.0).

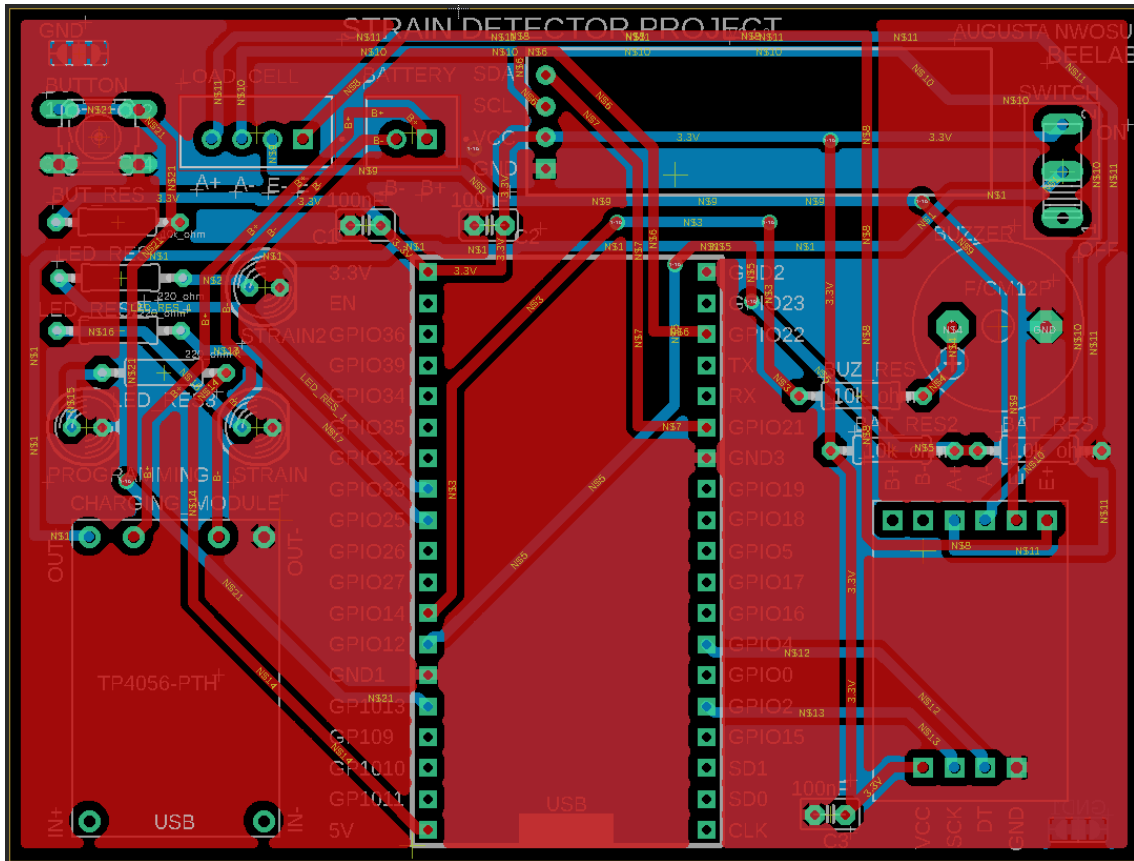


Figure 4: PCB copper pour/plane view illustrating ground fill and routing clearance.

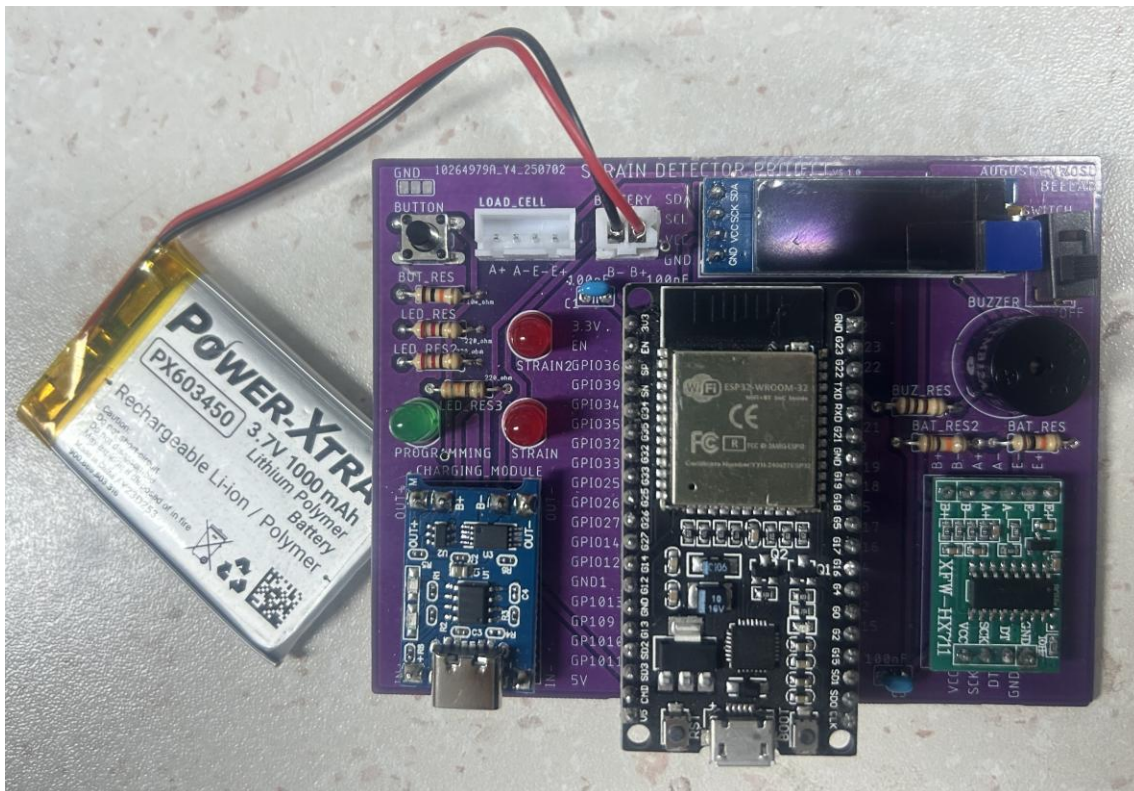


Figure 5: Assembled Device

Embedded firmware (Arduino IDE)

Firmware was written in the Arduino IDE for ESP32. The core functions of the device are data acquisition and averaging of raw HX711 readings, force conversion using a calibrated zero offset and scale factor, thresholding to detect clinically relevant pulling events, local alarm sequencing (timed buzzer beeps and LED activation), WiFi connection (including enterprise EAP support), and HTTPS posting of JSON alerts to the backend.

Design

1. Produced the complete UI/UX in Figma.
2. Defined the information architecture for clinicians: “WebSocket active” status, “Most at risk patient,” recent attempts table, and per-patient history panels.
3. Designed alert-first flows (banner + warning screen) with readable hierarchy for quick decisions during ward activity.
4. Established reusable components (cards, tables, dropdown accordions, alert modal) and responsive layout rules for desktop/tablet use.
5. Handed off design specs (spacing, typography, states, and interaction notes) to ensure correct implementation.

Front End Services

1. Built the dashboard UI in React/TypeScript using Next.js to display live patient status, attempt counts, and latest attempt timestamps.
2. Implemented alert interfaces that surface strain events immediately, including high-contrast warning views and user-dismiss controls.
3. Created client-side services to fetch historical records, populate tables, and render patient history sections on demand.
4. Implemented connection-state indicators (e.g., WebSocket active) and fault handling such as the reconnect logic.
5. Structured the front end into clean modules, components, API layer, state handling, to support clinical scalability.

Backend services

1. Developed the Node.js backend to receive device events via ESP32 HTTPS POST and standardize them into a consistent alert payload.
2. Implemented routes for: receiving alerts, querying patient history, returning “most at risk” summary, and recent activity feeds.
3. Added server-side validation using required fields, acceptable ranges, timestamping to prevent bad telemetry from corrupting logs.
4. Managed environment configuration with secrets, DB URI, deployment variables.
5. Designed the backend as the central hub between devices, database, and clinician dashboard.

WebSockets

1. Implemented a real-time WebSocket channel so clinicians see alerts instantly without page refresh.
2. Streamed events to all connected clients: patient name/ID, strain value, time of trigger, and alert state.

3. Built connection lifecycle handling: connect, heartbeat/keep-alive, reconnect strategy, and “active/inactive” UI status.
4. Ensured alert priority behavior (new events supersede stale views) and minimized latency from trigger → dashboard update.
5. Used WebSockets as the “live layer,” while REST endpoints provide history and summaries.

MongoDB logging

1. Integrated MongoDB as the persistent audit trail for clinical reviews and trial evidence.
2. Logged each alert attempt with structured fields. These fields are patient identifier, force/strain reading, time, device/source, and server receipt status.
3. Enabled query patterns used by the UI: most recent attempts list, per-patient history, weekly attempt counts, and “most at risk” ranking.
4. Added index-friendly document structure to keep reads fast as clinical trials scale (timestamps + patient keys).
5. Supported traceability: every on-device trigger can be matched to a stored record and a broadcast event to staff.

Analysis

Threshold-based detection in the firmware allows fast local actuation (audible/visual alarms) while simultaneously escalating events to clinical staff through the web platform. Using a persistent MongoDB store enables retrospective analysis such as frequency of attempts per patient, time-of-day patterns, and the effectiveness of interventions. Real-time WebSocket delivery minimizes alert latency and keeps the dashboard state consistent for all connected staff. During system verification, key checks included sensor repeatability after tare/zeroing, stability of force conversion, network resilience, and end-to-end integrity of alert records from device to database.

Conclusion

The patient strain detection and web-based monitoring system combines a custom strain-sensing hardware module with a secure, real-time web dashboard to reduce risk for restrained psychiatric patients. The author led the implementation across electronics, firmware, full-stack web development, and database-backed logging, delivering a practical workflow tool for clinicians and nurses. Ongoing clinical trials focus on validating alert thresholds, usability, and operational fit within the ward environment, with iterative refinement planned based on trial feedback.

Appendix A: ESP32 firmware

```
// Libraries definition
// 1. Display libraries
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// 2. Load Cell Amp library
#include "HX711.h"

// 3. Webserver & Wifi library
#include <Arduino_JSON.h>
#include <Arduino.h>
#include <WiFi.h>
#include <esp_eap_client.h>
#include <WebSocketsServer.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
```



```
// Wifi connection variables
const int timeout = 20000;
const char* ssid = "";
const char* username = "";
const char* password = "";

// Load cell amp module
int HX711_SCK = 2;
int HX711_DT = 4;

#define ZERO_OFFSET 5846
#define SCALE_FACTOR 64.58

// OLED Display declarations
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Passive components
const int buttonPin = 13;
const int LED_pin = 33;
const int LED_pin_2 = 25;
const int buzzer_pin = 14;
const int battery_pin = 12;

bool alarmActive = false;
unsigned long alarmStartTime = 0;
int alarmCycle = 0;
unsigned long lastBeepTime = 0;
bool isBeeping = false;

const float MAX_BATTERY_VOLTAGE = 3.3;
const float MIN_BATTERY_VOLTAGE = 2.2;

int buttonValue = 0;
JSONVar sensorData;

HX711 scale;
WebSocketsServer webSocket = WebSocketsServer(81);

void setup() {
  Serial.begin(115200);
  scale.begin(HX711_DT, HX711_SCK);

  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
  display.clearDisplay();

  // Connect to enterprise WiFi (EAP)
  WiFi.mode(WIFI_STA);
  esp_eap_client_set_identity((uint8_t*)username, strlen(username));
  esp_eap_client_set_username((uint8_t*)username, strlen(username));
  esp_eap_client_set_password((uint8_t*)password, strlen(password));
  esp_wifi_sta_enterprise_enable();
  WiFi.begin(ssid);

  while (WiFi.status() != WL_CONNECTED) {
    delay(50);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected! IP: " + WiFi.localIP().toString());

  webSocket.begin();

  pinMode(buttonPin, INPUT);
  pinMode(battery_pin, INPUT);
  pinMode(buzzer_pin, OUTPUT);
  pinMode(LED_pin, OUTPUT);
  pinMode(LED_pin_2, OUTPUT);
```

```
}

void loop() {
    websocket.loop();

    long raw = scale.read_average(10);
    float net = raw - ZERO_OFFSET;
    float force = net * SCALE_FACTOR;

    pullActivation(force);
    handleAlarm();

    buttonValue = digitalRead(buttonPin);
    if (buttonValue == 0) {
        pullActivation(25.0);
    }
}

void handleAlarm() {
    if (!alarmActive) return;

    unsigned long currentTime = millis();

    if (alarmCycle >= 5) {
        alarmActive = false;
        digitalWrite(LED_pin, LOW);
        digitalWrite(LED_pin_2, LOW);
        noTone(buzzer_pin);
        return;
    }

    digitalWrite(LED_pin, HIGH);
    digitalWrite(LED_pin_2, HIGH);

    if (!isBeeping && currentTime - lastBeepTime >= 200) {
        tone(buzzer_pin, 3000);
        lastBeepTime = currentTime;
        isBeeping = true;
    } else if (isBeeping && currentTime - lastBeepTime >= 300) {
        noTone(buzzer_pin);
        lastBeepTime = currentTime;
        isBeeping = false;
        alarmCycle++;
    }
}

int getBatteryPer() {
    int sum = 0;
    int battery_percentage = 0;

    for (int i = 0; i < 50; i++) {
        int rawADC = analogRead(battery_pin);
        float voltage = ((float)rawADC / 4095.0) * 3.3 * 2;

        if (voltage >= MAX_BATTERY_VOLTAGE) {
            battery_percentage = 100;
        } else if (voltage <= MIN_BATTERY_VOLTAGE) {
            battery_percentage = 0;
        } else {
            battery_percentage = ((voltage - MIN_BATTERY_VOLTAGE) / (MAX_BATTERY_VOLTAGE - MIN_BATTERY_VOLTAGE)) * 100.0;
        }

        sum += battery_percentage;
        delay(10);
    }

    battery_percentage = sum / 50;
    return battery_percentage;
}

void pullActivation(float force) {
    if (force > 13 && force < 100) {
        alarmActive = true;
    }
}
```

```
alarmStartTime = millis();
alarmCycle = 0;
lastBeepTime = 0;
isBeeping = false;

sensorData["name"] = "Jan";
sensorData["pressure"] = force;

String data = JSON.stringify(sensorData);

WiFiClientSecure client;
client.setInsecure();

HTTPClient https;
https.begin(client, "https://patient-strain-server.onrender.com/api/alert");
https.addHeader("Content-Type", "application/json");

int statusCode = https.POST(data);
String response = https.getString();

https.end();
}
}
```

Appendix B: HX711 calibration sketch

```
#include "HX711.h"
const int LOADCELL_DOUT_PIN = 4;
const int LOADCELL_SCK_PIN = 2;
HX711 scale;
float calibration_factor = 48400;

void setup() {
  Serial.begin(9600);
  scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN);
  scale.set_scale();
  scale.tare();

  long zero_factor = scale.read_average();
  Serial.print("Zero factor: ");
  Serial.println(zero_factor);
}

void loop() {
  scale.set_scale(calibration_factor);

  Serial.print("Reading: ");
  Serial.print(scale.get_units(), 1);
  Serial.print(" kg");
  Serial.print(" calibration_factor: ");
  Serial.print(calibration_factor);
  Serial.println();

  if(Serial.available()) {
    char temp = Serial.read();
    if(temp == 'a') calibration_factor += 1000;
    else if(temp == 'z') calibration_factor -= 1000;
    else if(temp == 'b') calibration_factor += 100;
    else if(temp == 'x') calibration_factor -= 100;
  }
  delay(1000);
}
```

Appendix C: Prototyped Handle

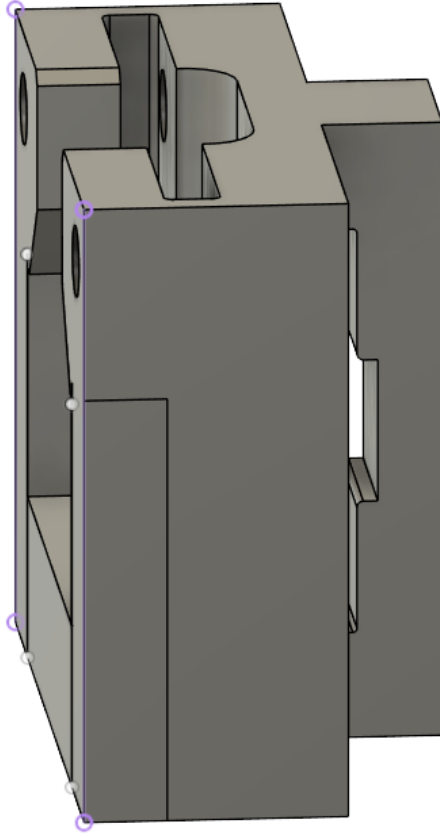


Figure 6: 3D Prototyped Handle For Restraint Strap

Project 2 - Title: Fatigue Detection System for Unmanned Aerial Vehicle (UAV) Pilot

Electronics Contribution: FSR Data-Acquisition Readout Circuit & PCB

Project No: FYL-2025-3898 (Ankara University BAP)

Abstract

This project proposed a contact-free, real-time fatigue monitoring system for UAV operators by combining posture-related pressure data with data from radar sensors and camera indicators. My contribution focused on the electronics of the pressure (FSR) subsystem. This entailed designing the analog readout circuit, selecting and finalizing components. This was for the signal conditioning stage. Next, I proceeded to create and verify a data-acquisition PCB that interfaces the FSR sensor array to an Arduino Due.

Introduction

Operator fatigue is a major safety risk in UAV operations as it is a high-attention, high-consequence domain. This risk is even amplified due to the long missions, repeated task cycles, and cognitive workload the drivers are subjected to. Another concern was the distortion or disturbance caused by contact-based sensors. To reduce the reliance on subjective reporting and contact-based sensors, the overall system combines three modalities:

- (1) seat/backrest pressure sensing using force-sensing resistors (FSRs) to quantify posture changes,
- (2) mmWave radar for contact-free cardiovascular/respiration tracking, and
- (3) camera-based facial behavior metrics.

Within this architecture, the pressure subsystem requires reliable data acquisition, which was the motivation for a custom-built and purpose-tailored PCB that conditions FSR signals and delivers them to the microcontroller for synchronized logging.

Methodology

1. System requirements and interface

The electronics were designed to:

- (i) acquire multiple FSR channels from the seat and backrest with stable, low-noise analog readings;
- (ii) interface cleanly to an Arduino Due (ADC inputs).
- (iii) support repeatable calibration/zeroing.

2. Readout circuit and signal conditioning

Each FSR channel was treated as a variable resistance sensor. The analog readings were gotten using the LM358 operational amplifiers. These amplifiers were selected for input buffering and signal amplification to improve signal integrity and reduce loading effects, enabling more reliable analog

capture at the Arduino Due ADC pins. The setup was created using the circuit below. This ensured conversion to a measurable voltage, followed by buffering and conditioning before digitization.

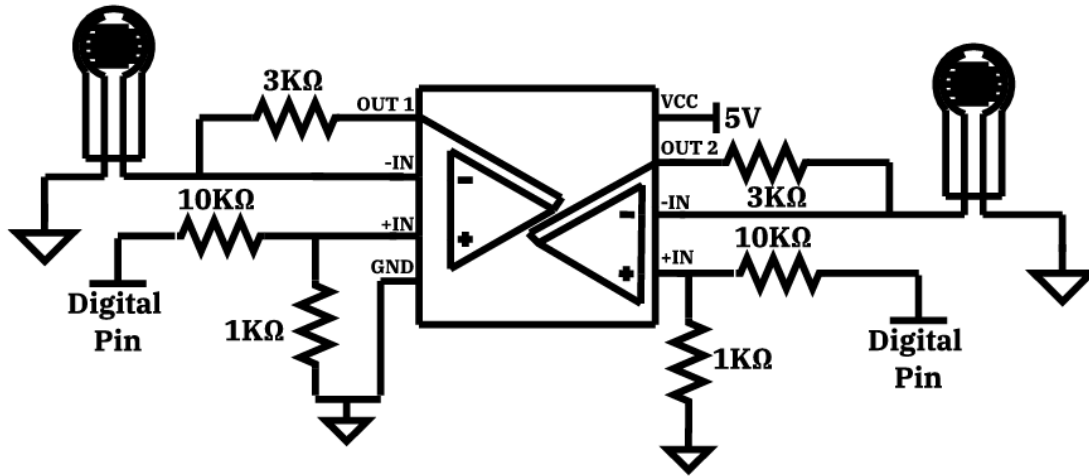


Figure 7: FSR Readout Circuit

3. PCB design and component finalization

A custom data-acquisition PCB was created using KiCAD to consolidate sensor connections and implement signal conditioning. The design contains dedicated connectors for each FSR channel, status LEDs, and control buttons supporting calibration and zeroing procedures. Early iterations reserved interface points for additional modules (accelerometer and galvanic skin response), but these were removed during optimization to focus the final board on the pressure-sensing subsystem.

Component list (Electronics & PCB)

Subsystem	Component	Role
Analog front-end	LM358 op-amp (x3 on PCB render)	Input buffering & signal conditioning for multiple FSR channels
Sensing	FSR 406 square sensors (x4)	Seat pressure array (2x2 placement)
Sensing	FSR 408 strip sensor (x1)	Backrest posture/leaning detection
MCU interface	Arduino Due	Analog capture of conditioned signals; synchronization of streams
User I/O	Tactile pushbuttons (e.g., RESET/TOGGLE/SPARE)	Calibration, zeroing, and control actions
User I/O	Status LEDs (Red/Yellow/Blue/Green)	Visual indication of system states
Passives	Resistors (e.g., 10k bias/pull, 330 ohm LED limit)	Biasing and LED current limiting (values visible on PCB render)
Passives	Capacitors (e.g., 1 uF)	Supply decoupling/signal smoothing (values visible on PCB render)

Connectivity	Sensor connectors (FSR, auxiliary headers)	PCB render) Rapid connect/disconnect during setup; clean cable routing
Mechanical	3D-printed enclosure	Secures PCB, minimizes vibration/movement, and exposes connectors externally

Schematic And PCB Design

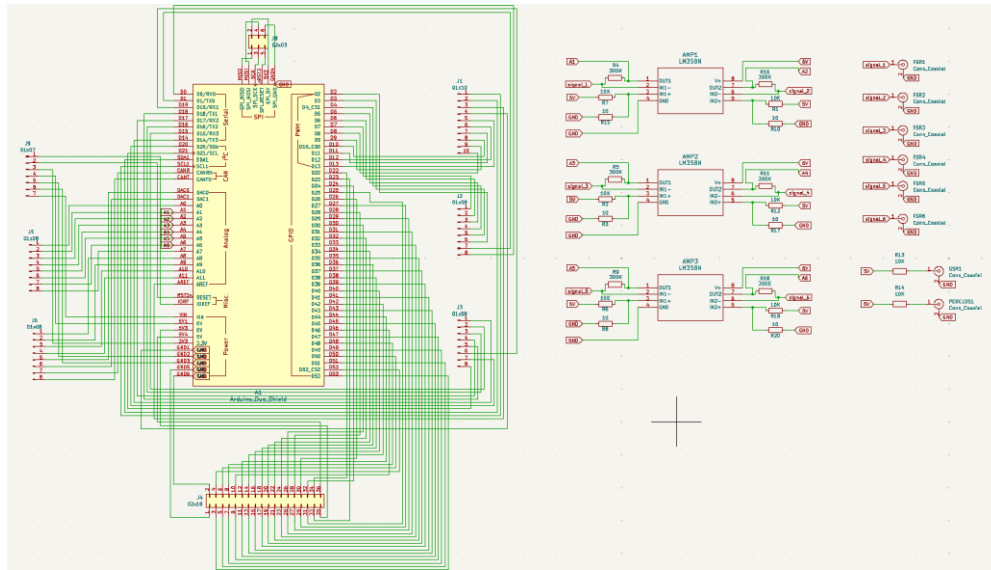


Figure 8: Schematic Design For The Device

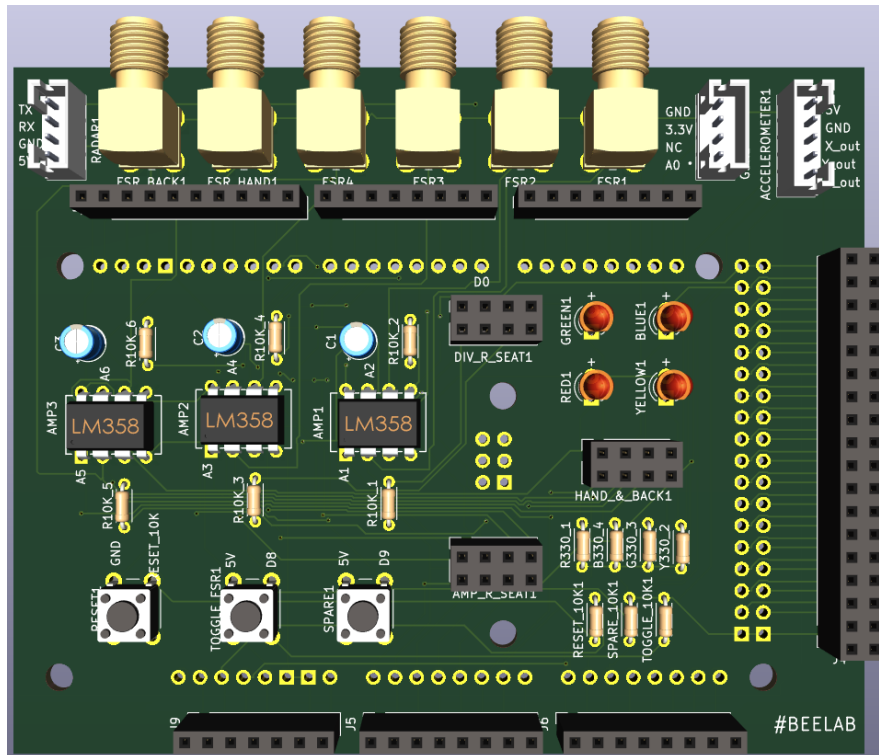


Figure 9: 3D PCB model and component placement for the FSR data-acquisition board.

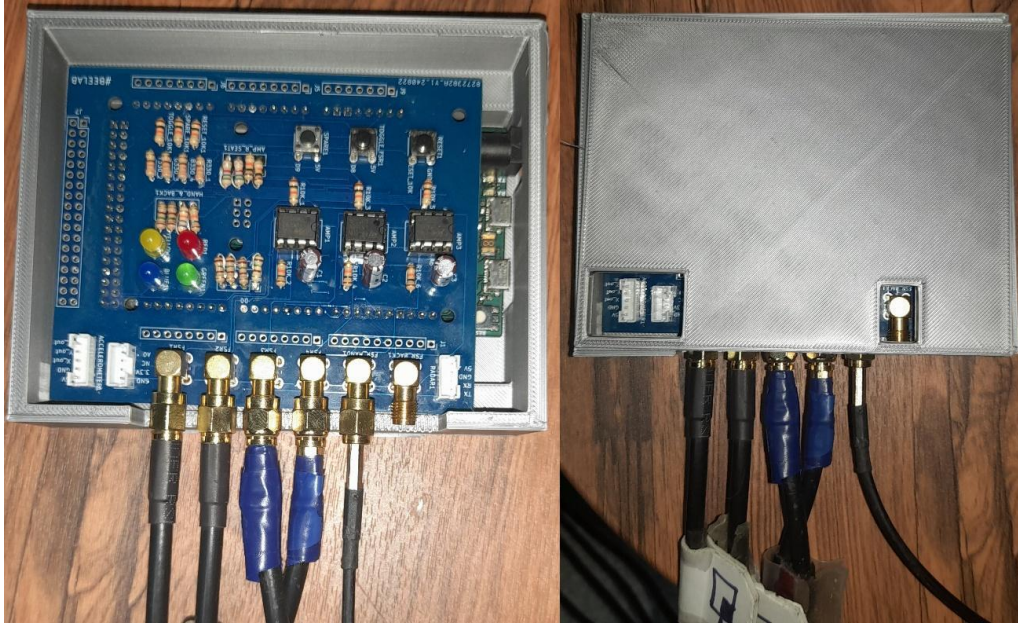


Figure 10: Data-acquisition PCB mounted in a custom enclosure (open and closed views) with externalized connectors.

4. PCB bring-up, testing, and verification

After fabrication and assembly, the PCB was verified through staged bring-up: basic continuity and power-rail checks, verification of connector pinouts, and functional validation of each conditioned analog channel at the Arduino Due inputs. Calibration/zeroing controls were done to confirm repeatable baseline setting before data collection. The mechanical enclosure was used to ensure stable operation with controlled cable routing and reduced movement during experiments.

Analysis

FSR sensors can produce high-impedance, noise-prone analog outputs. Also, the setup was constrained to run using the 5V terminal of the Arduino Due. Putting this into consideration, the resistors deployed used for amplification had their values tailored to these constraints. The LM358 op-amps reduce loading and help preserve signal fidelity from the seat/backrest sensors to the microcontroller ADC. Pushbuttons were integrated for calibration/zeroing, and multi-color LEDs for status feedback. This improves usability during live trials. The final board was deliberately simplified during optimization by removing planned interfaces for additional sensors (accelerometer/GSR), allowing the PCB area and routing to prioritize the pressure-sensing subsystem.

Conclusion

The electronics work delivered a purpose-built FSR data-acquisition solution: an analog readout and signal-conditioning approach, a finalized component set, and a fabricated and verified PCB that interfaces pressure sensors to an Arduino Due for synchronized logging within a multimodal fatigue monitoring system. By emphasizing signal integrity (buffering), repeatable operation (calibration/zeroing), and practical deployment (connectors, status indicators, enclosure), the board enabled reliable posture data capture as a key modality in the overall fatigue detection pipeline.

Project 3 - Title: Non-Contact Type Clinical Thermometer

Contribution: Electrical Engineer within a 7-Person Multidisciplinary Team

Abstract

A non-contact type clinical thermometer was developed as a multidisciplinary term project to provide rapid, hygienic body-temperature screening without direct skin contact. The system combines an infrared temperature sensor, an ESP32-based microcontroller, a compact OLED display, and an audible alert on a rechargeable battery platform. My responsibilities as the electrical engineer covered electronic component selection, circuit-level integration, PCB creation, power considerations, and support for verification and documentation. This report summarizes the electronics implementation approach, the configuration item list used for procurement and traceability, and key design considerations aligned with applicable requirements and standards.

Introduction

Non-contact thermometry reduces cross-contamination risk and can speed up basic triage in clinics, laboratories, and high-throughput screening environments. Unlike contact thermometers, an infrared-based thermometer estimates surface temperature from emitted radiation, enabling quick readings while maintaining a small stand-off distance. This project targeted a compact, battery-powered device with a simple user interface (single measurement trigger, on-screen readout, and optional buzzer indication) suitable for routine clinical workflows.

Project role and scope of electrical engineering work

Within the 7-person team, I served as the electrical engineer, responsible for ensuring that the sensing, power, user-interface, and embedded platform could be integrated into a reliable prototype. I:

- Selected and specified electronic components consistent with the system requirements and budget constraints.
- Integrated the infrared temperature sensing module with the microcontroller platform (I2C interfacing and electrical compatibility checks).
- Defined the power architecture (battery, charging module, switching, and safe interconnects) and verified stable operation under typical use.
- Supported prototype assembly, wiring, soldering, and hardware debugging during integration and testing cycles.
- Contributed to traceable documentation including the configuration item list, schematics/connection definitions, and design review inputs.

Methodology

System architecture overview

The device is organized around four main subsystems: (1) infrared sensing, (2) computation and control, (3) user interface, and (4) power/charging. The IR sensor provides measured object temperature data to the ESP32 microcontroller; firmware logic processes the reading and updates the OLED display. A button initiates measurement and a buzzer can signal threshold or warning

conditions. Power is supplied from a single-cell Li-ion battery with USB charging support via a dedicated charging module.

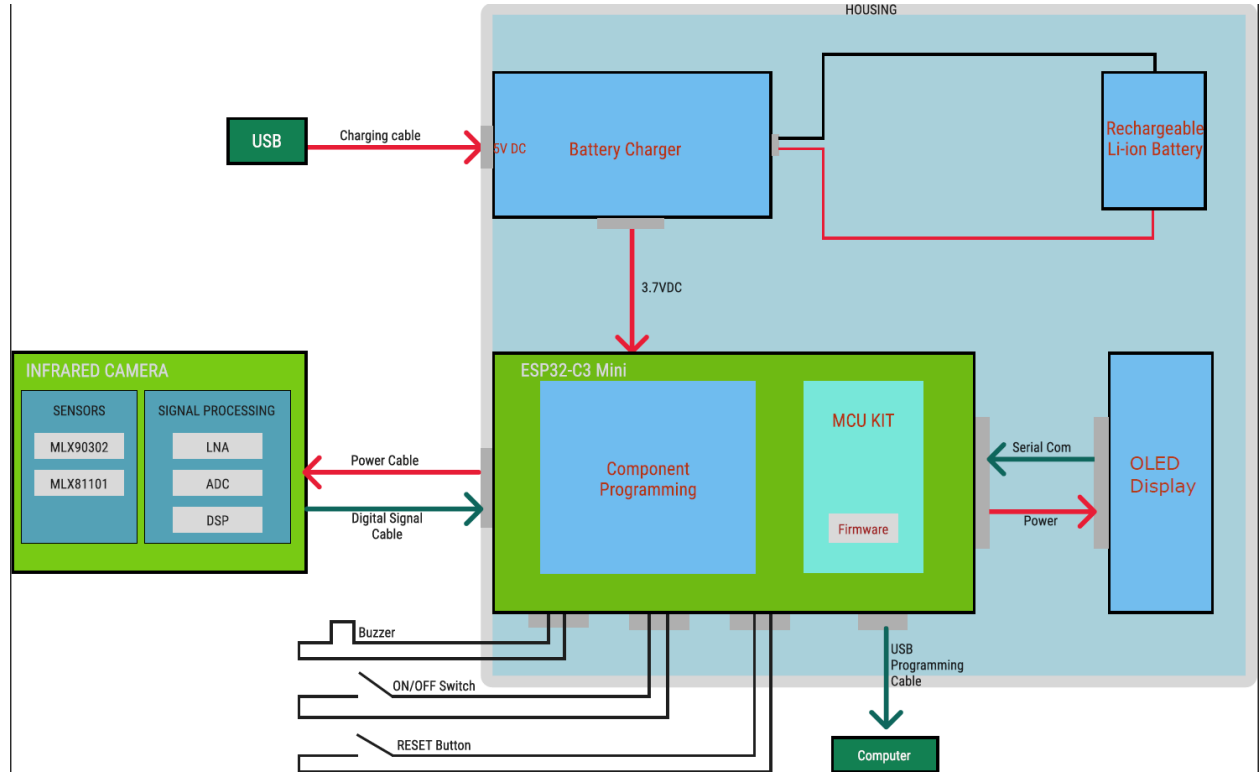


Figure 11: High-level hardware block diagram and interconnects.

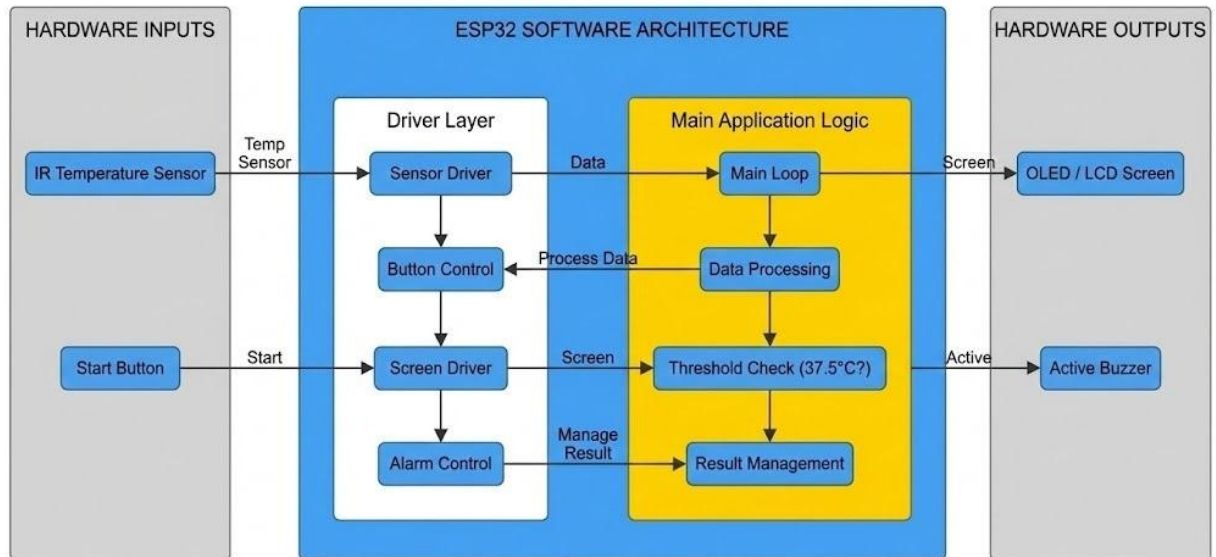


Figure 12: ESP32 software architecture diagram illustrating sensor, UI, and alarm control flow.

Electronics implementation details

The electronics implementation focused on the integration of the sensor, microcontroller, display, and alert peripherals while maintaining low complexity for assembly and debugging. This was essentially important as the printing process was done with a CNC machine present within the

laboratory of the department, hence the single layer design. Key electrical considerations included I2C bus integrity for the IR sensor and OLED module, GPIO assignment for the measurement button and buzzer, and a stable power path from the Li-ion battery through the charging and switching stages.

Component selection and configuration item list

A configuration item list was maintained to ensure traceability between the design and the procured components. Table 1 summarizes the primary hardware items used in the prototype.

Item No	Part Number	Description
1	01-PRT-001	MCU Kit, ESP32 C3 Mini
2	01-PRT-002	MLX90614, Infrared Camera Sensor
3	01-PRT-003	TP4056, Battery Charger Module
4	01-PRT-004	Display Kit, 0.96" LCD Display
5	01-PRT-005	On/Off Slider Switch
6	01-PRT-006	Measurement Button
7	01-PRT-007	Buzzer
8	01-PRT-008	Resistor
9	01-PRT-009	JST Connector
10	01-PRT-010	Lithium Ion Battery
11	01-PRT-011	Sensor PCB
12	01-PRT-012	Housing Assembly

Table 1. Configuration item list (primary electronics and mechanical items).

PCB Design And Verification

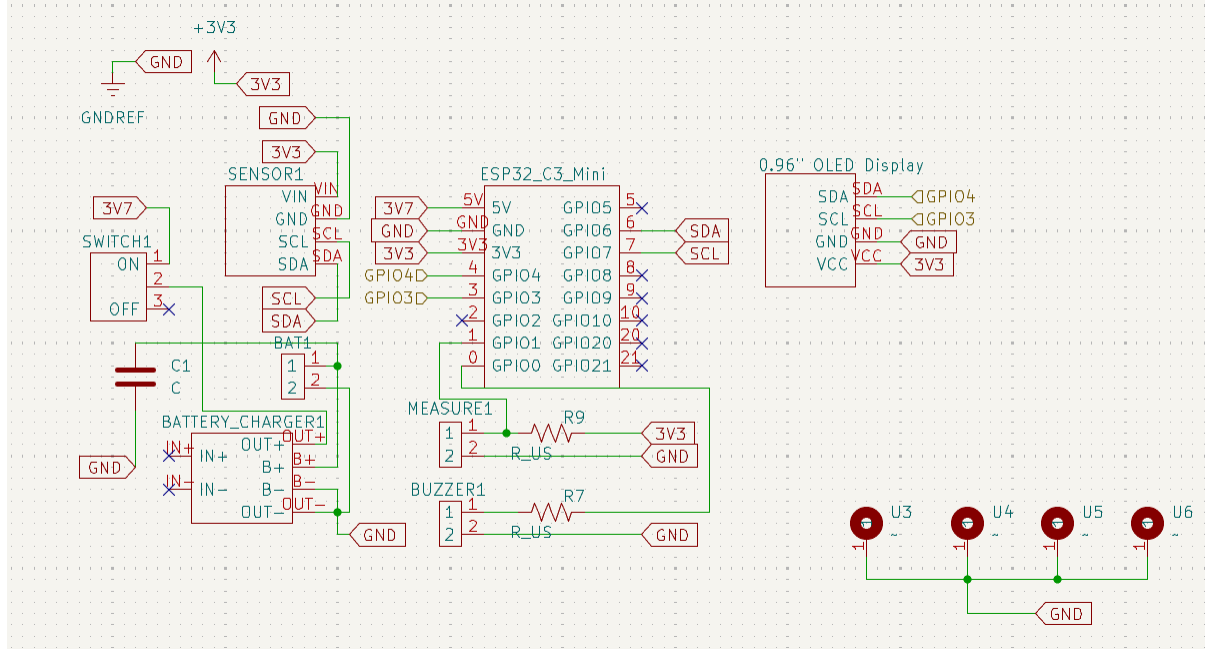


Figure 13: Image of the schematic diagram of the circuit board

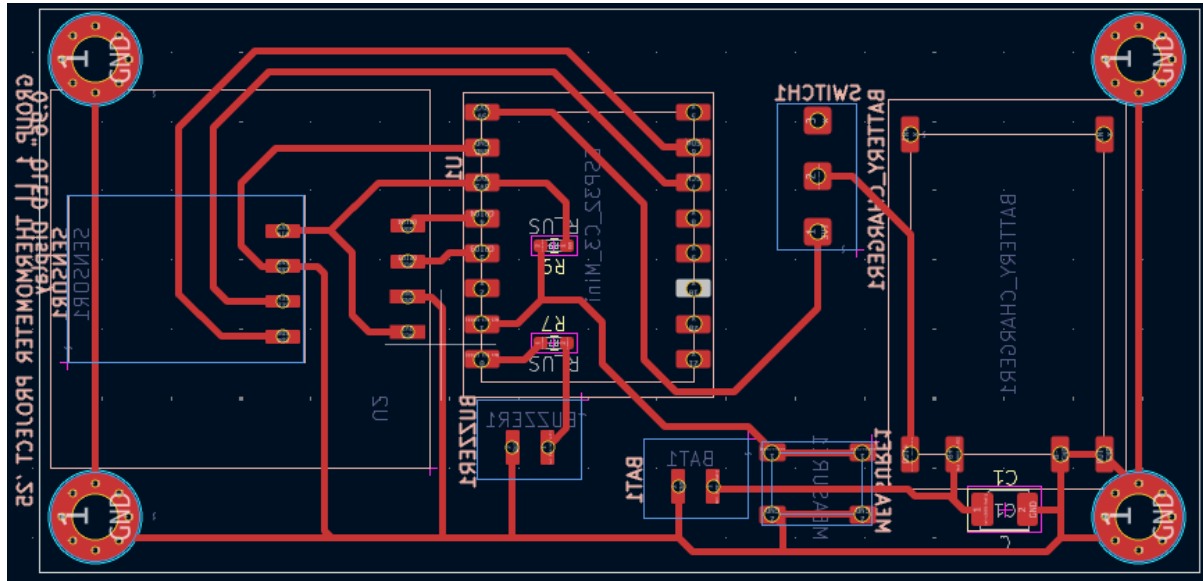


Figure 14: Printed Circuit Board showing single layer routing

Firmware-hardware interaction (summary)

Firmware on the ESP32 initiates an I2C read from the IR temperature sensor when the user triggers measurement. The returned temperature value is checked against acceptance limits and displayed on the OLED. The buzzer is activated for out-of-range or warning conditions.

Verification and testing approach

Electronics verification was performed through staged bring-up: power/charging validation, and end-to-end measurement checks. Power validation included confirming battery charging behaviour, switch operation, and stable supply for the microcontroller under display and buzzer load. Functional checks included I2C communication stability and repeatability of readings at the intended measurement distance.

Analysis

Key design trade-offs centered on sensor selection, power constraints, and usability. Infrared sensing enables fast, non-contact measurement, but performance depends on controlled distance, target emissivity, and ambient temperature drift. From an electronics perspective, I2C-based modules reduce PCB complexity, but require careful grounding and cable/trace management to avoid intermittent communication. Battery operation improves portability; however, it necessitates safe charging and predetermined low-power behavior to avoid sudden resets or corrupted readings. Our integration approach therefore prioritized stable power delivery, minimal wiring errors during assembly, and straightforward diagnostics via serial logging and modular testing.

Conclusion

This non-contact clinical thermometer project demonstrated a complete prototype pathway from requirements to integrated hardware bring-up within a multidisciplinary team. My work as the electrical engineer focused on converting system requirements into a practical electronics implementation: selecting components, integrating sensing and UI peripherals, validating power and charging behaviour, and supporting verification and documentation.

Project 4 - Title: Smart Wearable Rehabilitation Glove with Flexible Capacitive Sensors for Physiotherapy Progress Monitoring

Contribution: Project Lead & Head of Electronics || Ankara University Division

Parent Project: TUBITAK 1001 - Smart Object Recognition Glove Enhanced with Nanocomposite Capacitive Sensors

Abstract

This ongoing graduation project develops a smart, wearable rehabilitation glove that enables objective, real-time monitoring of hand therapy progress. The actualization of this project is a joint contribution between 2 university, Middle East Technical University and Ankara University, with the latter responsible for the actualization of the data acquisition electronics for the devices. The system uses flexible Ecoflex-based capacitive sensor arrays enhanced with high-permittivity fillers (BaTiO₃ and MXene) to achieve the mechanical compliance and sensitivity required for physiotherapy monitoring. Multi-channel electronics and synchronized acquisition are combined with inertial measurement (IMU) sensing to capture hand flexion, the distribution of pressure points, and kinematics during rehabilitation exercises. A custom microcontroller-based readout circuit is being developed with multiplexed acquisition, calibration support, and wireless data transmission.

Introduction

Hand rehabilitation after neurological or musculoskeletal injury depends on repeated, quantifiable exercises and ongoing evaluation, yet many therapy workflows still rely on subjective observation and direct reports from the patient. This project aims to provide clinicians with a wearable tool that measures therapy-relevant signals in real time for objective diagnosis. By combining flexible capacitive sensing with motion tracking, the glove is designed to support data-driven assessment of rehabilitation progress and enable consistent, evidence-based decision-making in both clinical and home-based settings.

Methodology

System Overview

The system proposed integrates flexible multi-channel capacitive sensors embedded within a wearable glove, an IMU for motion capture, a synchronized electronics readout unit with calibration and filtering, and a wireless link to a visualization dashboard for clinician review.

Engineering Phases

- Phase 1 – Sensor material selection and system definition:
Here, the comparative characterization of Ecoflex–BaTiO₃ composites (and reference dielectrics) using LCR measurements is done to create the capacitive sensor. A readout architecture is defined for prototyping, shielding, filtering, and power strategy selection.
- Phase 2 – Sensor fabrication and electronics prototyping:
In this phase, patterned capacitive sensors, tune fabrication parameters (filler ratio, thickness, curing, electrode adhesion), and bench-test a multi-channel readout prototype are created and monitored to measure noise, drift, hysteresis, repeatability, and response time.
- Phase 3 – PCB design and embedded system development:

A custom PCB is designed for synchronized multi-channel capacitive sensing with IMU integration; implement firmware for sampling, reference-capacitor calibration, digital filtering, and data packaging.

- Phase 4 – Wireless integration, visualization, and validation: implement wireless transmission for real-time monitoring; build a clinician visualization interface; validate the integrated glove through mock rehabilitation sessions and structured trials.

Electronics and Hardware Components (Planned)

The electronics stack is designed for multi-channel acquisition and wearable operation. It will also be wireless, optimized to have a long battery life. The parts include:

- A microcontroller platform is used for synchronized sampling and wireless communication.
- Capacitive-to-digital conversion and/or analog front-end options for capacitive sensing.
- Analog multiplexers to scale toward a 64-channel acquisition target.
- IMU for hand kinematics (flexion/motion tracking).
- Li-Po battery with power management and low-power operating modes.
- Electronics testing instrumentation: LCR meter, oscilloscopes, multimeters, and signal generators for validation.

PCB and Hardware Implementation

PCB schematics, layout, and board photographs will be appended after completion of Phase 3 (PCB design, verification, and fabrication). The PCB will provide channel routing, shielding/grounding strategy, reference capacitors for calibration, IMU integration, and battery-safe power management.

Roles, Duties, and Contributions

As project lead and head of electronics, I am responsible for delivering the end-to-end electronics subsystem and ensuring reliable acquisition. The data acquired must be suitable for use in rehabilitation.

- Readout circuitry: selection and prototyping of low-noise multi-channel capacitive readout approaches, including shielding and filtering strategies.
- Circuit testing: bench validation of sensitivity, signal-to-noise ratio, drift, hysteresis, repeatability, and timing synchronization under flexion/pressure cycles.
- PCB design and fabrication: translating the validated prototype into a manufacturable PCB, completing verification checks, and coordinating fabrication iterations.
- Power management: battery selection and regulation strategy using low-power dependent components.
- Embedded firmware: microcontroller programming for sampling, calibration (reference capacitors), digital filtering, and structured data acquisition/logging.
- Data acquisition: ensuring time-aligned multi-channel datasets suitable for processing in Python/MATLAB.

Analysis

Success Criteria

- Repeatable sensing across sessions with stable baselines and minimal drift.



- Adequate sensitivity and range for both subtle motions and firm grip forces.
- Synchronized multi-channel acquisition with reliable time alignment.
- Wireless reliability for real-time monitoring.
- Clinically relevant metric extraction (range of motion, grip profile, movement smoothness, repetition/consistency) and gesture recognition performance target above 85% accuracy.

Risks and Mitigation

- High noise in capacitive measurements: apply shielding, improve analog front-end stability, and add digital filtering.
- PCB delays or layout errors: use a design review checklist and rapid revision cycle.
- Synchronization issues: implement firmware buffering, deterministic sampling schedule, and timestamping.
- Wireless latency/instability: optimize configuration and support local edge buffering during dropouts.

Conclusion

This ongoing BEELAB graduation project aims to deliver a wearable rehabilitation glove that converts hand motion and contact pressure into objective therapy metrics.

Project 5 - Title: Design And Test of a Phonocardiogram System

Electronics Implementation & Testing Contribution (Multidisciplinary Project Work, 2024–2025)

Abstract

This report is focused on documenting the electronic contribution to the Ankara University Multidisciplinary Project Work titled “Design and Test of a Phonocardiogram System,” which aimed to build a practical, testable phonocardiogram device by combining a custom stethoscope head, an analog signal-conditioning circuit, and software-based monitoring and processing. My responsibilities centered on hardware implementation: component selection, circuit prototyping, soldering and assembly of the provided PCB design, validation of the signal-conditioning stages, and iterative testing using both the recommended transducer (speaker-as-microphone) and an electret microphone alternative. The final system using the recommended microphone produced recordable heart-sound waveforms through a laptop microphone input while the electret microphone did not.

Introduction

Phonocardiography (PCG) captures acoustic vibrations generated by the heart and translates them into an electrical signal that can be amplified, filtered, recorded, and analyzed. In the 2024–2025 Multidisciplinary Project Work at Ankara University, teams from Biomedical, Computer, Software, and Electrical and Electronics Engineering collaborated to design and test a complete phonocardiogram system, spanning stethoscope fabrication, analog front-end electronics, monitoring software, and signal processing demonstrations.

Methodology

System overview

The system includes 4 major parts:

- (1) stethoscope head and acoustic coupling,
- (2) microphone/transducer conversion,
- (3) analog signal-conditioning (gain + band-pass filtering + output scaling), and
- (4) recording/monitoring on a host device (e.g., laptop).

Although the full course project includes a software deliverable, the scope here focuses on the stethoscope assembling and electronics that enable a clean, recordable PCG signal.

Electronics implementation

The analog front-end was implemented around an LM324N quad op-amp, using three stages: a first-stage amplifier to boost the low-level microphone signal, a multiple-feedback band-pass filter stage to emphasize the heart-sound band while removing the out-of-band noise, and a final gain stage to scale the signal to the recording computer. The circuit was assembled on the provided PCB design (PCB layout was not designed by me), and all through-hole components were soldered and inspected for solder bridges, cold joints, and polarity errors by me.

Schematic And PCB Design

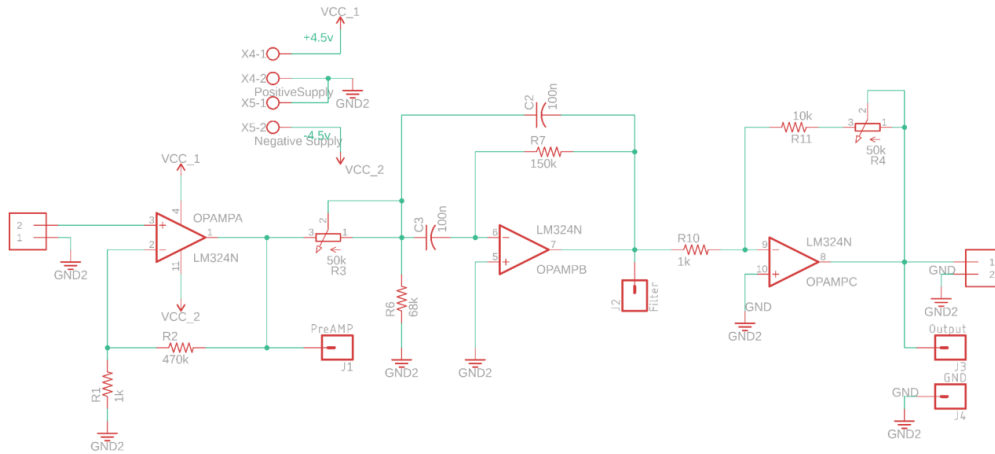


Figure 15: Signal-conditioning schematic (LM324N-based amplifier and filter stages) – not made by me

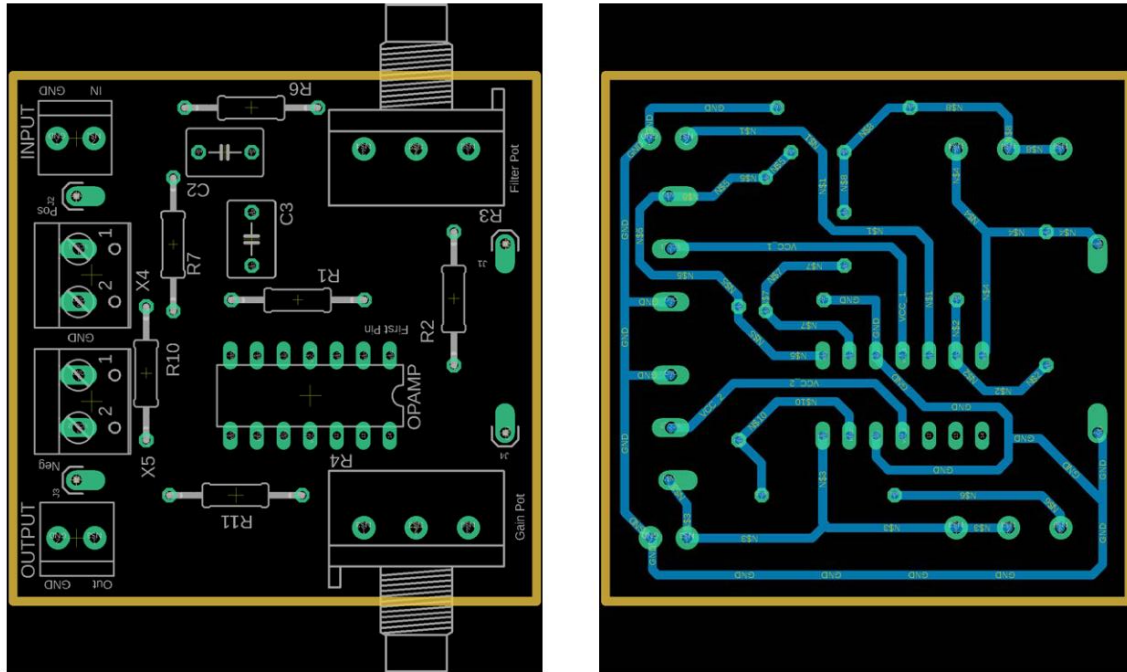


Figure 16: PCB layout views used for assembly and wiring verification (top and bottom layers) not made but assembled by me.

Microphone options and coupling

Two microphone approaches were evaluated during implementation: (i) the recommended speaker-as-microphone transducer, and (ii) an electret. For the speaker-as-microphone configuration, the input coupling capacitor present on the PCB design was intentionally bypassed/shorted (jumper) to match the recommended build approach. For the electret microphone option, the coupling capacitor was installed (and biasing was verified) to ensure proper AC coupling into the first amplification stage. The potentiometer settings were calculated, computed and tuned to achieve stable gain and filtering without clipping at the recorder input.

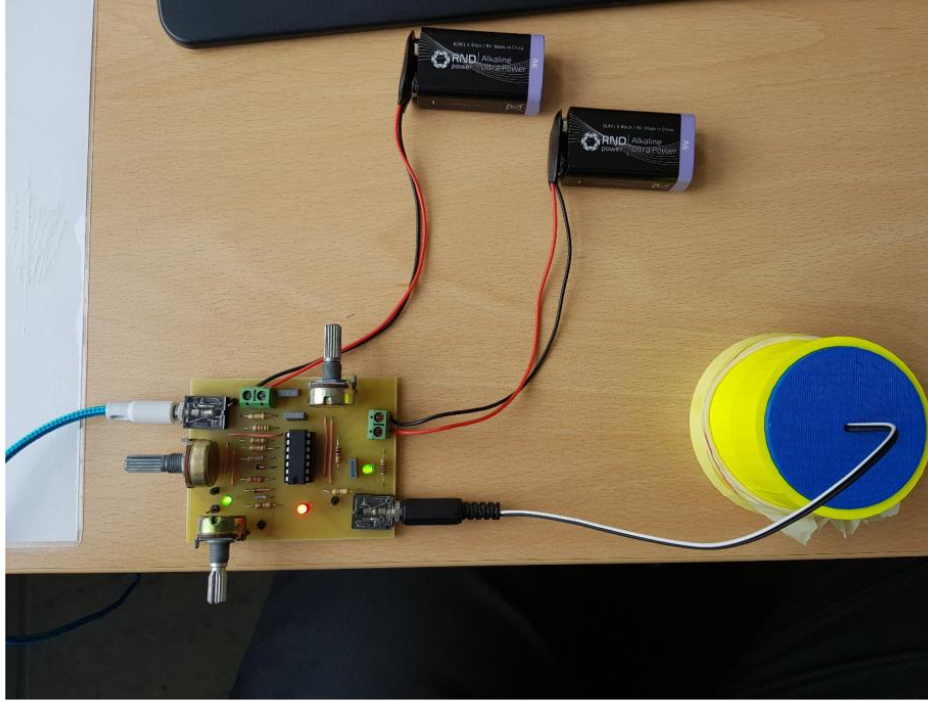


Figure 17: Physical stethoscope-to-board connection and assembled hardware context.

Components and bill of materials

A simplified bill of materials for the electronics assembly is provided below (quantities reflect a single PCB build).

Component	Value/Type	Qty	Notes
Op-amp IC	LM324N (DIP-14)	1	Quad op-amp used across the analog chain
IC socket	DIP-14 socket	1	Recommended for easier replacement/debug
Resistor	1 k Ω	2	Gain-setting / bias network
Resistor	10 k Ω	1	Gain-setting / bias network
Resistor	68 k Ω	1	Filter/gain network
Resistor	150 k Ω	1	Filter/gain network
Resistor	470 k Ω	1	Filter/gain network
Potentiometer	50 k Ω	2	Tuning gain / filter conditioning
Capacitor	100 nF	2	Filter capacitors (per design)
Terminal block	2-pin (3.5 mm)	2	External connections (as specified)
Terminal block	2-pin (5.0 mm)	2	External connections (as specified)
Battery clip	9 V battery connector	1	Power input



Cables	3.5 mm audio jack cable	1	Connection to host PC/laptop input
Transducer	Speaker (as microphone)	1	Primary acoustic pickup
Alternative transducer	Electret microphone capsule	1	Tested alternative configuration

Role Summary

- Electronics implementation lead for the project portion: I handled component selection, circuit build, soldering, inspection, and test.
- Transducer evaluation: I validated the recommended speaker-as-microphone build and tested an electret microphone alternative with appropriate coupling.
- Signal conditioning tuning: I computed and adjusted potentiometer settings to achieve stable gain and reduce saturation/clipping during recordings.
- System-level verification: I captured and reviewed PCG recordings to confirm expected temporal periodicity and low-frequency spectral characteristics.

Assembly and verification workflow

- Reviewed the schematic and PCB layout to map component reference designators to physical placement.
- Added passive components first (resistors, capacitors, potentiometers) to the PCB, then connectors and IC socket, and finally the op-amp IC.
- Applied standard soldering quality checks (joint wetting, fillet shape, avoidance of bridges), and reworked joints when necessary.
- Validated continuity on power rails and critical nets, then performed a powered smoke-test with current-limited supply/battery.
- Checked amplifier output for clipping and baseline offset; iteratively tuned potentiometers to stabilize gain and improve signal-to-noise.

Analysis

Signal capture and qualitative validation

After assembly, the system was tested by coupling the stethoscope head to the chest area and recording the output through a laptop microphone input. A representative recording (provided as a WAV sample) was inspected in both the time domain and frequency domain to confirm the presence of periodic heart-sound bursts and a concentration of energy in the expected low-frequency band.

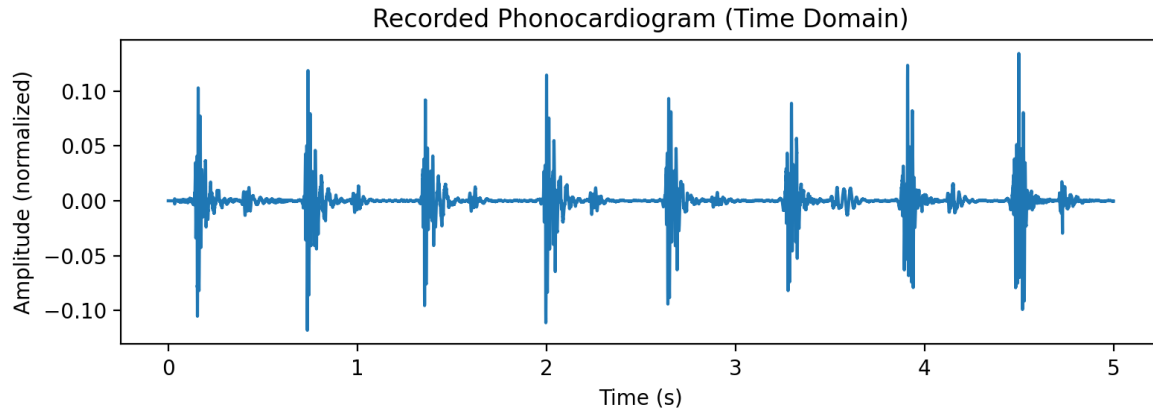


Figure 18: Example PCG recording in the time domain (normalized amplitude).

Electronics-focused observations

- Input transducer choice materially affects required coupling and biasing: the speaker-as-microphone configuration can operate with a bypassed coupling capacitor, while the electret microphone requires appropriate coupling and bias verification.
- The electret microphone used wasn't suitable for low amplitude sounds such as the heart beat.
- Potentiometer tuning enabled practical control of gain and conditioning during real-world tests, especially to prevent clipping at the recorder input.
- PCB assembly quality (solder joints, connector strain relief, and grounding continuity) strongly influences noise pickup.

Conclusion

This project demonstrated an end-to-end phonocardiogram hardware build that converts heart sounds into a recordable electrical signal suitable for basic monitoring and signal-processing exercises. My contribution focused on the electronics implementation: assembling and validating the LM324N-based signal-conditioning circuit on the provided PCB, selecting and testing components, comparing speaker and electret microphone configurations (including the required coupling choices), and performing iterative tuning and recording checks.